

Build an Interview-Ready Backend Project

Build one small service deeply enough to explain its decisions and limitations. A booking, task, or inventory service can demonstrate the essentials without pretending to be a global platform. Keep the scope small, use maintained libraries, and make behavior reproducible with synthetic data.

For: Engineers who want a project that demonstrates contracts, persistence, tests, failure handling, and operational reasoning rather than a large collection of unfinished features.

Suggested pace: 4 weeks with a narrow product scope | Basic API and database skills

What you will be able to do

- Define a minimal user workflow and a clear API/data contract.
- Implement persistence, validation, authorization, and meaningful tests.
- Add one reliability feature with explicit failure behavior.
- Present a reproducible demo and a defensible architecture narrative.

Build the right mental model

Scope is an engineering choice

Choose one primary workflow, such as creating and completing a task. Write non-goals and stop adding features until the core path is correct. A small service with recovery tests demonstrates more depth than many disconnected endpoints.

Use proven building blocks

Choose one framework, database, migration tool, and test framework appropriate to your language. Do not write custom authentication or a homegrown database to make the project appear advanced. Explain integration and trade-offs instead.

Evidence makes a project credible

Keep tests, sample requests, schema, configuration instructions, and limitations visible. A local benchmark must state hardware and workload. Distinguish implemented behavior from a future design proposal and personal work from copied tutorial scaffolding.

Study with evidence.

Work in weekly releases with a usable result at each boundary. Keep a short issue list, commit coherent changes, and run focused checks after every behavioral edit. Spend time on documentation and failure tests rather than only new features.

Week 1: Contract and first path

Define entities, state transitions, ownership, errors, and one complete workflow. Implement a minimal API and database schema with migrations. Add request validation and a small fixture dataset.

Deliverable and check: A reproducible create/read workflow and tests for malformed and missing inputs.

Week 2: Correctness and access

Add update/delete behavior where needed, server-side authorization, uniqueness constraints, and concurrency tests. Define retries and idempotency for a selected write. Keep secrets outside source and logs.

Deliverable and check: A contract suite covering ownership, duplicate requests, conflicts, and a concurrent update.

Week 3: Reliability and operations

Add one background task or external integration using a mocked provider. Implement bounded retries, visible status, structured logs, and graceful shutdown. Measure one performance property under a documented workload.

Deliverable and check: A restart or timeout test, an operational dashboard or log query, and a measured result with caveats.

Week 4: Package and explain

Containerize or deploy through a suitable simple platform, validate setup from scratch, and write an architecture decision note. Rehearse a ten-minute walkthrough including one failure and one rejected alternative.

Deliverable and check: A shareable demo, clear README, tests, and an honest description of what production hardening remains.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

A retryable task-processing service

The API accepts a task, a worker transforms a small synthetic input, and the client polls for status. A worker can crash, and the client can retry a creation request after losing its response.

A worked approach

1. Define stable task and request identities, accepted/queued/running/completed/failed states, and input size limits. Store the task and operation result transactionally so duplicate creation returns the same logical task.
2. Have the worker claim work with a defined lease or transaction boundary, persist completion, and acknowledge only afterward. If processing has an external effect, give that effect a separate idempotency or reconciliation strategy.
3. Test a crash before processing, after output persistence, and before acknowledgement. Verify that status is recoverable, accepted work is not silently lost, and a stale worker cannot overwrite a newer terminal state.

Expected result

The project demonstrates a meaningful lifecycle rather than only CRUD endpoints. The README can explain durable state, retries, ownership, and recovery using concrete tests and a small understandable architecture.

Verify the result

- Does a repeated creation request return the same task?
- Can a new worker recover abandoned work safely?
- Does the demo run with synthetic data and no private credentials?

Extend the exercise

Add cancellation and define its boundary: queued work may be cancelled immediately, while in-flight work may require cooperative cancellation. Report the outcome explicitly rather than claiming every operation can be interrupted safely.

Make the learning stick.

How many technologies should the project use?

Only those needed to demonstrate the chosen contract. A clear framework/database combination with good tests is often stronger than unnecessary microservices, brokers, and clusters that you cannot explain or operate.

Should I call it production-ready?

Only if the claim is justified by the actual security, reliability, operations, and support work. For most learning projects, state implemented features and known gaps instead of making an unsupported blanket claim.

What should the demo emphasize?

Show the primary user workflow, one important failure case, tests, and a design trade-off. Explain what you built and how you verified it. A long tour of every file is usually less useful than one coherent request path.

Avoid these traps

- Expanding scope before the main workflow is correct.
- Inventing scale or production usage for a local demo.
- Leaving setup, secrets, and recovery behavior undocumented.

Ready for the next step?

- A new reader can run the project from the README.
- Contracts and state transitions have meaningful tests.
- One failure path is demonstrated and explained.
- Claims distinguish implemented behavior from future ideas.

Reliable next reads

- [The Twelve-Factor App](#)
- [OWASP API Security](#)
- [GitHub README guidance](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.